Книга 2

Промпт-инжиниринг

Как формулировать запросы к ИИ: от базовых принципов до few-shot, Chain-of-Thought и структурированного вывода

Начинающий — Продвинутый

Библиотека ИИ · Книга 2 из 5

Содержание

1. 1. Что такое промпт и почему он важен
2. 2. Базовые принципы хорошего промпта
3. 3. Роль и контекст: system prompt
4. 4. Few-shot prompting: обучение на примерах
5. 5. Chain-of-Thought: пошаговое рассуждение
6. 6. Структурированный вывод (JSON, таблицы)
7. 7. Продвинутые техники: self-consistency, ReAct, tree-of-thought
8. 8. Антипаттерны: чего не делать
9. 9. Шаблоны промптов для типовых задач
10. 10. Практикум

1

Что такое промпт и почему он важен

Текст, который вы даёте модели — это и есть промпт

Промпт — это входной текст, который вы отправляете языковой модели. От того, как он сформулирован, напрямую зависит качество ответа. Один и тот же вопрос, заданный по-разному, может дать блестящий ответ или совершенно бесполезный.

Аналогия

Промпт — как заказ в ресторане. «Принесите еды» — повар приготовит что угодно. «Стейк средней прожарки, без гарнира, соус на стороне» — получите именно то, что хотите.

Пример: плохой и хороший промпт

Плохо:

```
"Напиши о маркетинге."
```

Модель не знает: для кого, какой формат, какой тон, какой объём. Результат будет общим и скучным.

Хорошо:

```
«Напиши пост для Telegram-канала (до 500 символов) о трёх преимуществах email-маркетинга для малого бизнеса. Тон: дружелюбный, профессиональный. В конце — призыв подписаться на рассылку.»
```

Конкретика даёт модели всё необходимое для точного ответа.

Ключевые выводы

- Промпт = входной текст для модели
- Качество промпта определяет качество ответа
- Конкретика и контекст — ключевые факторы

2

Базовые принципы хорошего промпта

Шесть правил, которые работают всегда

Принцип: Будьте конкретны

Чем точнее опишите задачу, тем лучше результат. Укажите формат, длину, тон, целевую аудиторию.

Принцип: Дайте контекст

Модель не знает вашей ситуации. Объясните: для кого это, зачем, какие ограничения.

Принцип: Укажите формат

Хотите список? Таблицу? JSON? Абзац? Скажите это явно.

Принцип: Задайте роль

«Ты — опытный маркетолог» или «Ты — преподаватель физики» — это сужает стиль и уровень ответа.

Принцип: Ограничьте длину

«В 2-3 предложениях» или «не более 200 слов» — иначе модель может растекаться.

Принцип: Разбейте сложное на шаги

Вместо одного огромного промпта — несколько последовательных, каждый решает одну часть задачи.

Мнемоника: К.Ф.Р.О.Д.К.

Контекст, Формат, Роль, Ограничения, Действие, Конкретика. Если промпт содержит все шесть элементов — он почти наверняка хороший.

Ключевые выводы

- Конкретика, контекст, формат, роль, ограничения, шаги
- Мнемоника К.Ф.Р.О.Д.К. для проверки промпта
- Разделяйте сложные задачи на несколько промптов

3

Роль и контекст: system prompt

Невидимая инструкция, задающая тон всему диалогу

System prompt (системный промпт) — это инструкция, которая задаёт поведение модели для всего разговора. В API она передаётся отдельно от пользовательских сообщений.

```
Системный промпт:
"Ты — опытный преподаватель программирования для начинающих. Объясняй просто, с примерами
на Python. Избегай жаргона. Если ученик ошибается — мягко поправляй."
```

Что включать в system prompt

- Роль и экспертизу модели
- Тон и стиль общения
- Ограничения (чего не делать)
- Формат ответов
- Контекст задачи (для RAG-систем — источник данных)

> ⚠ **Важное отличие**
>
> System prompt задаёт «правила игры» на весь диалог, а пользовательский промпт — конкретный запрос в рамках этих правил. Плохой system prompt не спасёт даже идеальный пользовательский промпт.

Ключевые выводы

- System prompt = глобальная инструкция для всего диалога
- Включает роль, тон, ограничения, формат
- В API передаётся отдельно от пользовательских сообщений

4

Few-shot prompting: обучение на примерах

Покажите модели, что вы хотите — она повторит паттерн

Few-shot prompting — это техника, при которой вы даёте модели несколько примеров вход → выход, и она продолжает в том же стиле. Нулевой промпт (zero-shot) — без примеров, один промпт (one-shot) — с одним примером.

Пример few-shot для классификации тональности

```
Промпт:
Текст: "Отличный сервис, очень доволен!" → Тональность: Позитивная
Текст: "Ужасное качество, не рекомендую." → Тональность: Негативная
Текст: "Доставили вовремя, ничего особенного." → Тональность: Нейтральная
Текст: "Лучший опыт покупки за год!" → Тональность:
```

Модель продолжит паттерн и ответит «Позитивная».

Сколько примеров нужно?

Количество	Когда использовать
Zero-shot (0)	Простые задачи, модель уже умеет
One-shot (1)	Нужно показать формат ответа
Few-shot (3-5)	Нужна стабильность формата и стиля
Many-shot (10+)	Сложные паттерны, редкие задачи

Совет

Располагайте примеры от простых к сложным. Это помогает модели постепенно «понять» паттерн. И обязательно включайте разнообразные примеры, а не только один тип.

Ключевые выводы

- Few-shot = даём примеры вход → выход
- Zero/one/few-shot — выбор зависит от сложности задачи
- Примеры должны быть разнообразными и идти от простых к сложным

5

Chain-of-Thought: пошаговое рассуждение

Заставьте модель думать вслух — результат лучше

Chain-of-Thought (CoT) — это техника, при которой модель побуждают рассуждать шаг за шагом, прежде чем дать окончательный ответ. Это особенно полезно для логических и математических задач.

Без CoT

```
Промпт: "В магазине 23 яблока. 12 продали, 6 испортились и их выбросили. Потом привезли
ещё 15. Сколько осталось?"
Ответ: 20
```

С CoT

```
Промпт: "Реши задачу пошагово.
В магазине 23 яблока. 12 продали, 6 испортились и их выбросили. Потом привезли ещё 15.
Сколько осталось?"

Шаг 1: 23 - 12 = 11 (осталось после продажи)
Шаг 2: 11 - 6 = 5 (осталось после выбрасывания)
Шаг 3: 5 + 15 = 20 (стало после поставки)
Ответ: 20
```

Варианты CoT-промптов

- «Думай пошагово» — простейший вариант
- «Реши задачу по шагам, показывая каждый шаг» — более развёрнутый
- «Сначала разбей задачу на части, потом реши каждую» — декомпозиция
- «Проверь свой ответ, перечитав задачу» — самопроверка

Когда CoT особенно полезен

Математика, логика, программирование, анализ с несколькими условиями, правовые рассуждения. Для простых задач (перевод, суммаризация) CoT не нужен и может даже замедлить.

Ключевые выводы

- CoT = модель рассуждает по шагам, потом даёт ответ
- Снижает ошибки в логических и математических задачах
- «Думай пошагово» — достаточно для активации

6

Структурированный вывод (JSON, таблицы)

Когда нужен машинно-читаемый результат

Если ответ модели дальше обрабатывается программой, нужен структурированный формат. JSON — самый распространённый выбор.

Промпт для JSON-вывода

```
Промпт:
"Извлеки информацию из текста и верни ТОЛЬКО JSON в формате:
{
"имя": "...",
"возраст": 0,
"профессия": "...",
"город": "..."
}
Не добавляй ничего кроме JSON.

Текст: «Ивану 32 года, он работает инженером в Новосибирске.»"
```

```
Ответ:
{
"имя": "Иван",
"возраст": 32,
"профессия": "инженер",
"город": "Новосибирск"
}
```

Советы для надёжного структурированного вывода

1. Скажите «верни ТОЛЬКО JSON» — без пояснений до и после
2. Покажите шаблон с примером заполнения
3. Укажите типы данных (строка, число, массив)
4. Если поле необязательное — скажите «может быть null»
5. Для критичных систем парсите ответ и валидируйте схему
6. Используйте «режим JSON» если API его поддерживает

> ⚠ **Гарантия формата**
>
> Даже с лучшим промптом модель иногда добавит лишний текст. В продакшене: используйте режим JSON (если есть), парсите ответ через json.loads() и валидируйте поля. При ошибке — повторите запрос.

Ключевые выводы

- JSON-вывод требует явного шаблона и запрета лишнего текста
- «Верни ТОЛЬКО JSON» — ключевая фраза
- В продакшене — парсинг + валидация + повтор при ошибке

7 Продвинутые техники

Self-consistency, ReAct, Tree-of-Thought

Self-Consistency

Модель генерирует несколько вариантов рассуждения (с CoT) по одному промпту, а потом выбирает наиболее частый ответ. Повышает точность на логических задачах.

```
Шаг 1: сгенерировать 5 разных рассуждений (temperature=0.7)
Шаг 2: собрать финальные ответы
Шаг 3: выбрать наиболее частый (голосование)
```

ReAct (Reasoning + Acting)

Модель чередует рассуждение и действия: подумала → вызвала инструмент → получила результат → подумала снова. Так работают ИИ-агенты с доступом к поиску, калькулятору, API.

```
Thought: Мне нужно узнать текущий курс доллара
Action: search("курс доллара сегодня")
Observation: 1 USD = 95.2 RUB
Thought: Теперь посчитаю: 100 USD = 9520 RUB
Action: calculate(100 * 95.2)
Observation: 9520
Answer: 100 долларов = 9520 рублей
```

Tree-of-Thought (ToT)

Модель исследует несколько веток рассуждения одновременно, оценивает каждую и выбирает лучшую. Похоже на дерево решений, но для рассуждения. Эффективно для задач планирования и поиска.

Ключевые выводы

- Self-consistency: несколько рассуждений + голосование
- ReAct: чередование рассуждения и действий (агенты)
- Tree-of-Thought: исследование веток рассуждения

8

Антипаттерны: чего не делать

Ошибки, которые портят результат

Антипаттерн: Слишком размытый промпт

«Расскажи что-нибудь интересное» — модель не знает, что вы хотите. Результат случаен.

Антипаттерн: Слишком длинный промпт

Промпт на 3 страницы с вложенными условиями — модель теряется. Разбейте на части.

Антипаттерн: Противоречивые инструкции

«Будь краток, но подробно расскажи про...» — модель не знает, что важнее.

Антипаттерн: Ожидание идеального ответа с первой попытки

Промпт-инжиниринг — итеративный процесс. Уточняйте, переформулируйте, просите исправить.

Антипаттерн: Игнорирование контекста

«Переведи это» без указания языка, стиля, целевой аудитории — перевод будет, но, возможно, не тот.

Антипаттерн: Копирование чужих промптов без адаптации

Чужой промпт может работать для чужой задачи. Адаптируйте под свою.

Ключевые выводы

- Размытые и слишком длинные промпты — главные ошибки
- Противоречия в инструкциях путают модель
- Промпт-инжиниринг — итеративный процесс, не одноразовый

9

Шаблоны промптов для типовых задач

Готовые структуры, которые можно адаптировать

Задача: Суммаризация

```
Сделай краткое резюме следующего текста в {N} предложениях. Сохрани ключевые факты и цифры. Текст: {ТЕКСТ}
```

Задача: Генерация контента

```
Напиши {ТИП_КОНТЕНТА} на тему «{ТЕМА}» для {АУДИТОРИЯ}. Тон: {ТОН}. Объём: {ОБЪЁМ}. Включи: {ЭЛЕМЕНТЫ}.
```

Задача: Анализ данных

```
Проанализируй следующие данные и найди 3 ключевых инсайта. Для каждого инсайта: что обнаружено, почему это важно, что рекомендуешь. Данные: {ДАННЫЕ}
```

Задача: Извлечение информации

```
Извлеки из текста все упоминания: имен, дат, сумм, организаций. Верни в формате таблицы. Текст: {ТЕКСТ}
```

Задача: Объяснение для новичка

```
Объясни {ПОНЯТИЕ} так, будто рассказываешь {УРОВЕНЬ_АУДИТОРИИ}. Используй аналогию из повседневной жизни. Не более {N} абзацев.
```

Задача: Код-ревью

```
Проверь следующий код на {Python}. Найди: баги, проблемы производительности, нарушения стиля. Для каждой проблемы: строка, описание, исправление. Код: {КОД}
```

Ключевые выводы

- Шаблоны ускоряют работу, но требуют адаптации
- Фигурные скобки { } — места для подстановки ваших данных
- Сохраняйте удачные шаблоны в библиотеку для повторного использования

10

Практикум

Задания для самостоятельной работы

Задание 1: Улучшение промпта

Дан промпт: «Напиши о пользе спорта.» Перепишите его, используя все шесть принципов (К.Ф.Р.О.Д.К.). Сравните результаты обоих промптов в любой модели.

Задание 2: Few-shot классификация

Составьте few-shot промпт для классификации отзывов на 3 категории: «продукт», «доставка», «сервис». Проверьте на 5 отзывах.

Задание 3: Chain-of-Thought

Возьмите 3 задачи из школьной программы по математике. Сравните ответы с CoT и без CoT. Запишите, где CoT помог, а где — нет.

Задание 4: JSON-извлечение

Напишите промпт, который извлекает из описания товара: название, цену, категорию, характеристики (массив). Проверьте, что ответ парсится через json.loads().

Задание 5: Свой шаблон

Выберите задачу из вашей работы/учёбы. Создайте шаблон промпта по структуре из главы 9. Протестируйте на 3 разных входных данных. Доработайте шаблон по результатам.

Конец Книги 2. Следующая — Книга 3: Архитектура нейросетей и трансформеров.